SecureForge

ROUND 1 READINESS DRAFT - live-model demo and video pending

Team

Aditya Ranjan (@kvianAR) and Shubhaang Kataruka (@S-h-u-b-h-1). Team name, Team ID and each member's college or employer are pending confirmation. Roles cover Build, Defend and Prove with shared integration review.

Starting point

Custom SecureForge extension and tools built from scratch, using the official TrueForge SDK and runtime. This branch extends the shared SecureForge core-agent foundation. It does not fork TrueForge. The executable agent definition is src/agent.mjs. Public repository github.com/S-h-u-b-h-1/SecureForge.

Architecture on TrueForge

The VS Code client creates TrueForge sessions and streams turns through SDK 0.182.0 against local runtime 0.183.0. It registers one temporary authenticated loopback MCP connector per run. The model chooses read_file, search_workspace, lookup_symbol, discover_repository, propose_edit, propose_batch and separately approved run_command calls. Provider credentials stay in TrueForge.

TrueForge approvals gate proposal staging. The extension then renders immutable hash-linked proposals for independent human hunk decisions. One WorkspaceEdit applies accepted batch changes to buffers. Guarded Undo checks versions and hashes. Save is explicit. Cancellation discards pending proposals and closes the bridge. Command approval is separate, shell execution is disabled and receipts are bounded and redacted.

How context is selected

Automatic lexical retrieval scores task terms in paths and source lines, then boosts related tests and documentation. Explicit file/symbol mentions and the inline selection's file receive priority. Each excerpt shows path, lines, hash and selection reason. Default serialized context budget is 24,000 UTF-8 bytes, adjustable from 2,000 to 96,000. At most 12 excerpts are selected after scanning at most 4 MiB across 2,500 candidates. Token counts are approximate bytes/3 estimates.

Repository excerpts are marked untrusted. Reads exclude restricted names, symlinks, binary data and oversized files. The integrated discovery MCP tool adds lexical import hints with a separate 160-file default limit. Lexical lookup does not infer callers or AST relationships. A pinned Flask rehearsal retrieved CLI source, tests and documentation without explicit filenames. A real-model recording must still demonstrate its usefulness.

Model setup

The current TrueForge catalog is empty. Production-model A/B runs and provider cost measurement were skipped. Runtime/SDK/MCP/editor checks used a deterministic local inference fixture, which does not prove model reasoning. Configure a provider in TrueForge, then set secureforge.model to its model FQN. Example settings and agent spec are included. Actual model, provider location and measured cost must be entered after the official run.

Scorecard

Every video timestamp is pending. The evidence below describes local checks and implementation status. It does not assign competition points.

Criterion	Video time	Current evidence and remaining demonstration
A1 Inline edit	Pending	Selection guard and per-hunk review implemented. Show official Task A in the editor.
A2 Agent mode	Pending	2-8-file staging supported. Autonomous three-file Task B discovery requires a real model.
A3 Streaming and cancellation	Pending	Runtime stream and cancellation safety tested. Show responsive editing during generation.
B1 Context assembly	Pending	Budgeted retrieval, paths and reasons implemented. Flask source/test/doc retrieval checked.
B2 Diff safety	Pending	Real editor apply, per-hunk rejection and guarded Undo verified. Show these on screen.
B3 Resilience (Task C)	Pending	Actual SDK unreachable-endpoint check preserves Flask hashes and Git status. No automatic fallback or backoff.
B4 TrueForge integration depth	Pending	SDK sessions, authenticated MCP and approvals verified with fixture inference. Advanced harness feature demonstration pending.
C1 Video	Pending	A/B/C storyboard prepared. The required four-minute editor recording has not been created.
C2 Write-up	Pending	Eight headings included. Team metadata, real model/cost and video timestamps must be completed.

Tools used

Codex assisted implementation, tests and documentation. Node.js, the official TrueForge SDK/runtime, VS Code, esbuild and vsce provide the extension workflow. Python 3.12, pytest 8, Flask 3.1.3, ReportLab and Poppler support rehearsal and document checks. No production-model API cost was measured.

Known limitations and what you would do with another month

The fixed Flask edits were manually specified reference changes. Task A false-value checks and 59 Task B CLI tests passed, including masked JSON and help registration. This evidence does not establish autonomous official-task success. Task C has local SDK failure evidence, while editor responsiveness still needs recording. Team details, the final video, final scorecard and portal submission remain open.

Security findings use four narrow patterns plus bounded same-function Python request-input traces. They remain candidates until controlled proof supports them. Exploitability proof is limited to the owned traversal fixture. Redaction is best effort. New-file edits, persistent proposals, automatic model fallback, complete authorization analysis and a demonstrated advanced TrueForge context-engineering feature remain future work.

With another month, prioritize real-model evaluations, recording and scoring the official tasks, AST-aware context and input tracing, transaction-safe new-file review, and measured harness compaction or deferred tool loading. Preserve human edit approval and evidence boundaries throughout.